PyTorch and CUDA



I: installing CUDA

II: run your code on GPU

PyTorch and CUDA



I: installing CUDA

II: run your code on GPU

PyTorch and CUDA

problem: even for moderate setups, computational time becomes the limiting factor

self made ANN → keras/tensor flow → pytorch → pytorch on GPU (Cuda)

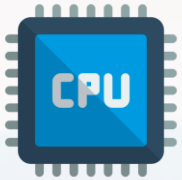
speed

```
512/512 [==============================] - 4072s 8s/step - loss: 3.4534 - accuracy: 0.3338
Epoch 4/5
512/512 [==============================] - ETA: 0s - loss: 3.1926 - accuracy: 0.3980     saved  ../data/segmentation
pics/checkpoints//.3
512/512 [==============================] - 3363s 7s/step - loss: 3.1926 - accuracy: 0.3980
Epoch 5/5
512/512 [==============================] - ETA: 0s - loss: 3.0071 - accuracy: 0.4318   saved  ../data/segmentation
pics/checkpoints//.4
512/512 [==============================] - 3616s 7s/step - loss: 3.0071 - accuracy: 0.4318
```

Lenovo T14, NVIDIA GeForce MX450: (simple LSTM)

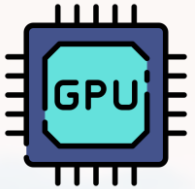
Keras (CPU):	**300 sec**
PyTorch (CPU):	11 sec
PyTorch (GPU):	**3 sec**

PyTorch and CUDA



training AI

→ mainly matrix operations
→ GPUs are a lot better at it!





CUDA is the link of your GPU to Python (PyTorch)

check, if graphic card is on list

check your graphics device:

→ Windows command shell prompt
or **environment command shell**

→ type `nvidia-smi`

→ press *Enter*

```
(base) PS C:\Users\MMH_user> nvidia-smi
Tue Mar 17 20:36:51 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 573.91                 Driver Version: 573.91         CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                  Driver-Model | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce MX450           WDDM |   00000000:01:00.0 Off |                  N/A |
| N/A   71C    P0              N/A  /    9W |       0MiB /   2048MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                      GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

PyTorch and CUDA

1) set up CUDA environment
2) install libraries
3) check if it worked
4) test in your UI

Watch **`Install_Cuda.mp4`**

PyTorch and CUDA

1) **set up CUDA environment**
2) install libraries
3) check if it worked
4) test in your UI

It makes sense to set up a **specific environment** in order to install **specific versions** of the required packages!

```
conda create -n MyCUDAenv python=3.11
```

```
(base) PS C:\Users\MMH_user> conda create -n MyCUDAenv python=3.11
```

don't install the most recent version in order to avoid version conflicts!

```
conda activate MyCUDAenv
```

```
(base) PS C:\Users\MMH_user> conda activate MyCUDAenv
```

```
(MyCUDAenv) PS C:\Users\MMH_user>
```

PyTorch and CUDA

It makes sense to set up a **specific environment** in order to install **specific versions** of the required packages!

1) set up CUDA environment
2) **install libraries**
3) check if it worked
4) test in your UI

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
```

Even if my GPU could run on `cuda 12.8`, we install an older version in order to avoid version conflicts!

```
(MyCUDAenv) PS C:\Users\MMH_user> conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
```

PyTorch and CUDA

We check if it all worked!

1) set up CUDA environment
2) install libraries
3) **check if it worked**
4) test in your UI

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

```
(MyCUDAenv) PS C:\Users\MMH_user> python -c "import torch; print(torch.cuda.is_available()); print(torch.version.cuda)"
```

```
True
12.1
```



```
python -c "import torch; print(torch.cuda.get_device_name(0))"
```

```
(MyCUDAenv) PS C:\Users\MMH_user> python -c "import torch; print(torch.cuda.get_device_name(0))"
```

```
NVIDIA GeForce MX450
```



PyTorch and CUDA

1) set up CUDA environment
2) install libraries
3) check if it worked
4) **test in your UI**

We check if it all worked!

→ Install and open your favorite UI

CheckMyCuda.py

```python
import torch

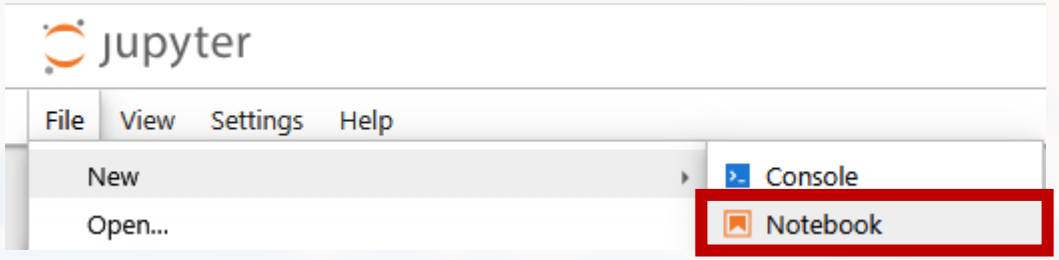
def test_cuda():
    print("PyTorch version: ", torch.__version__)
    print("CUDA version: ", torch.version.cuda)
    print("CUDA Available: ", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("Number of GPUs: ", torch.cuda.device_count())
        print("GPU Name: ", torch.cuda.get_device_name(0))

if __name__ == "__main__":
    test_cuda()
```

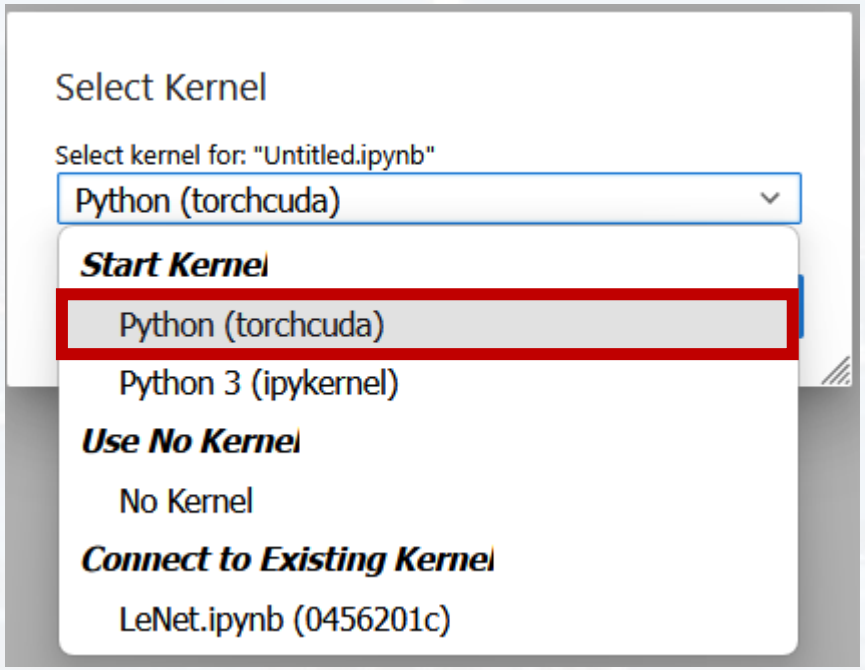


```
PyTorch version:  2.5.1
CUDA version:  12.1
CUDA Available:  True
Number of GPUs:  1
GPU Name:  NVIDIA GeForce MX450
```

PyTorch and CUDA



1) set up CUDA environment
2) install libraries
3) check if it worked
4) **test in your UI**




```python
import torch

def test_cuda():
    print("PyTorch version: ", torch.__version__)
    print("CUDA version: ", torch.version.cuda)
    print("CUDA Available: ", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("Number of GPUs: ", torch.cuda.device_count())
        print("GPU Name: ", torch.cuda.get_device_name(0))

if __name__ == "__main__":
    test_cuda()
```

```
PyTorch version:  2.5.1
CUDA version:  12.1
CUDA Available:  True
Number of GPUs:  1
GPU Name:  NVIDIA GeForce MX450
```

PyTorch and CUDA



I: installing CUDA

II: run your code on GPU

PyTorch and CUDA

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

Congratulation! If you see this, you are ready to go!

PyTorch and CUDA

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

Torch objects like **`model`** or **`torch.tensor`** have the property **`.to`**

```
TrainX = torch.tensor(TrainX, dtype = torch.float32)
TrainY = torch.tensor(TrainY, dtype = torch.float32)
```

turning `numpy array` into `torch.tensor`

```
TrainX = TrainX.to(device)
TrainY = TrainY.to(device)
```

allocating objects to the device

```
model  = model.to(device)
```

PyTorch and CUDA

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

When running the training, we need to **synchronize** between GPU (for training the model) and CPU (for everything else)…

```
torch.cuda.synchronize()

        #training loop
        for epoch in range(n_epochs):
                outputs = model(TrainX)

                ... #do stuff...

torch.cuda.synchronize()
```

PyTorch and CUDA

The key part in PyTorch is to set all matrices and the model **to the device (CPU or GPU)**

```
In [13]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ...: print("Using device:", device)
Using device: cuda
```

When running the training, we need to **synchronize** between GPU (for training the model) and CPU (for everything else)…

…and later allocate the model back to CPU

```
PredY = model(TestX).detach().to('cpu').numpy()
```

Check out `GAN architecture.ipynb` as an example

PyTorch and CUDA

Happy Coding!

